

JUEGUITO v0.0.1

Creación de funciones de “Output” e “Input” para texto.

INTRODUCCIÓN

En cualquier tipo de juego, siempre habrá la necesidad de mostrar texto de forma eficiente o, incluso, de recibir las respuestas del usuario con una función de “Input” o entrada de datos. Un ejemplo bastante claro de esto es el juego de Undertale, donde prácticamente todo se detalla a través del texto más que por el ambiente o diseño visual. También, el juego y su sistema de combate se desarrollan a través de dos distintas opciones predefinidas: El Ataque y El Acto, los cuales son las dos responsables de direccionar la historia hacia donde el jugador desee. Para nuestro proyecto versión 0.0.1, se desarrollarán funciones predefinidas parecidas para ahorrarnos todo el trabajo repetitivo que este tipo de sistemas conlleva.

OBJETIVO

Desarrollar una función ***void Output()***, responsable de escribir todo el texto plano mediante una variable que contenga todo el texto a mostrar, y otra ***int Input()*** que dependa de números enteros para escoger y tengan límites de opción que no nos permita escoger ninguna otra opción a las dadas.

PASOS A SEGUIR

Primero, hay que crear el archivo ***Juego.cpp*** donde se programará todo el juego. Dentro, se agregará la estructura fundamental, siendo estructurada de la siguiente manera:

```
#include <iostream>
```

```
#include <string>
```

```
int main(){  
    return 0;  
}
```

Teniendo la estructura básica, las funciones ***void Output()***, ***int Input()*** y, en general, todas las demás funciones creadas, se crearán entre las librerías (en este ejemplo siendo la última ***#include <string>***) y la función ***int main(){...}***.

Las funciones deberán de tener argumentos específicos para que cumplan su función correctamente:

--- FUNCIÓN OUTPUT ---

La función ***Output()*** deberá contener por el momento sólo un argumento llamado ***Texto*** de tipo ***std::string***, proveniente de la anterior librería ***<string>***. Este tipo de variable se declara como cualquier otra, ya sea ***int***, ***float***, etc.; ***std::string*** es un tipo de dato que almacena cadenas de texto. Aquí es donde se introducirá el texto para poder imprimir cualquier estilo de texto.

--- FUNCIÓN INPUT ---

La función `Input()` tendrá, en este caso, tres argumentos fijos, los cuales serán *int LimInferior*, *int LimSuperior* y *std::string Pregunta*. Las variables `LimInferior` y `LimSuperior` se utilizarán dentro de un ciclo `do{...} while()` para evitar que el jugador escoja opciones fuera del límite establecido. Por ejemplo: si las opciones para alguna pregunta son 1, 2 y 3, no queremos que el jugador escoja cualquier otra opción que no sea estos números; aquí es donde entran los límites, donde el `LimInferior` será 1 y el `LimSuperior` será 3. Si el jugador se sale de los límites, el ciclo `do{...} while()` no dejará que la función termine hasta que escoja un número válido dentro del límite establecido.

En el caso de la variable `Pregunta`, se utilizará para declarar las opciones disponibles mediante texto. Por ejemplo: si hay tres opciones para alguna batalla “**1. Atacar | 2. Actuar | 3. Curar:** ”, el texto `Pregunta` deberá contener estas opciones. Este texto se implementará dentro del ciclo `do{...} while()`, en la parte más superior antes de preguntar qué se quiere escoger, para que siempre se repita con cada ciclo que pase.

Dentro de la función, se declarará una variable extra *int Opcion*. Esta nueva variable será la encargada de preguntarle al jugador qué es lo que se quiere escoger mediante la función *std::cin*, encargada de recibir datos. Esta se pondrá después del texto impreso.

La estructura de la función *Input()* debería verse así:

```
int Input(argumentos){
int Opcion;
do{
    Imprimir texto (Pregunta);
    Recibir opción (Opcion);
}while(LimInferior && LimSuperior);
Regresa Opcion;
}
```

CRITERIOS A SEGUIR

1. Para que el código sea legible para cualquier persona, las líneas de código anidadas deberán estar separadas por sangrías bien definidas, pudiéndose colocar mediante la tecla **tab** en el teclado, justo como se muestra en los ejemplos.
2. Todas las variables extras que se puedan necesitar deberán tener un nombre claro.
3. Si no se sabe muy bien desde la lectura cómo funciona una función específica, agrega comentarios `//` para hacerle saber a los demás programadores para qué sirve cada función o variable dentro de las mismas.
4. Intenta comentar con tus otros compañeros programadores si tienes alguna duda o si no sabes si una función está bien definida.